

Perceiver & Perceiver IO

Implementation and Experimental Analysis

Francesco Gigli and Corso Vignoli

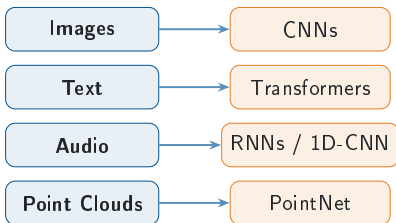
Università degli Studi di Firenze
Course B031278 — Deep Learning

Agenda

- 1 Introduction
- 2 Perceiver
- 3 Experiments: Perceiver
- 4 Perceiver IO
- 5 Experiments: Perceiver IO
- 6 Conclusion

The Problem: Modality-Specific Architectures

One modality, one architecture



Each data type is usually paired with a dedicated model and domain-specific assumptions about structure.

The scaling bottleneck

Self-attention cost

$$\mathcal{O}(M^2 \cdot d)$$

grows quadratically with input length M

Input representation	M
16×16 image patches	256
raw 224×224 pixels	50,176

Raw pixels make standard attention
impractical.

Core question: can one architecture handle arbitrary inputs without quadratic cost?

The Perceiver Solution

Core idea: learned latents read the input once; expensive processing stays in latent space.

1. Read

Cross-attention

M inputs $\rightarrow N \ll M$ latents

2. Process

Latent block

transform N latent slots

3. Reuse

Shared weights

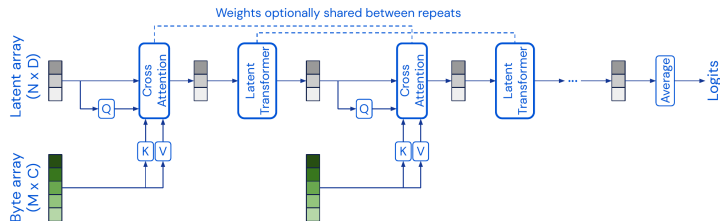
repeat the same processor

Complexity shift

Read the full input once; repeat the expensive work in the compact latent space $N \ll M$.

Operation	Cost
Read input once	$\mathcal{O}(M \cdot N)$
Process latents for L layers	$\mathcal{O}(L \cdot N^2)$
Perceiver	$\mathcal{O}(MN + LN^2)$

Perceiver Architecture



Encode and process

- $X \in \mathbb{R}^{M \times C}$ supplies keys and values.
- $Z \in \mathbb{R}^{N \times D}$ supplies queries, with $N \ll M$.
- Cross-attention maps $X \rightarrow Z$: $\mathcal{O}(MN)$.

Latent core and output

- Latent Transformer updates Z only.
- Repeated blocks may share weights.
- Original Perceiver pools Z ; Perceiver IO uses output queries.

Cross-Attention: The Key Mechanism

Formally:

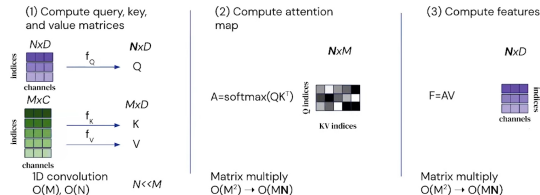
$$Q = Z W_Q \in \mathbb{R}^{N \times d_k}$$

$$K = X W_K \in \mathbb{R}^{M \times d_k}$$

$$V = X W_V \in \mathbb{R}^{M \times d_k}$$

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

The **latents Z are the queries**, the input X provides keys and values $\Rightarrow$ informational *bottleneck*.

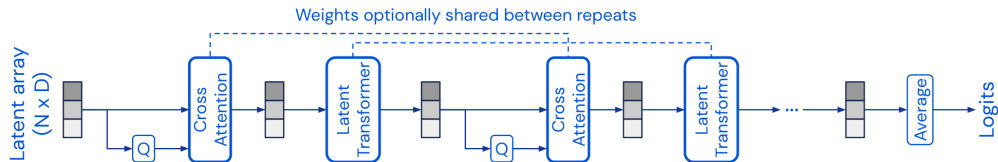


Key insight:

- Attention matrix: $N \times M$ (not $M \times M$)
- Complexity: $\mathcal{O}(N \cdot M \cdot d_k)$
- Decouples depth from input size

Latent Transformer & Weight Sharing

Once information is in the latent array, depth is spent by reusing the same processor on a compact state.



Weight sharing

- The same Latent Transformer block B_θ is reused at every repeat, with shared parameters θ .
- Unrolling depth behaves like an RNN over the latent state: $Z_{t+1} = B_\theta(Z_t)$.

Why it matters

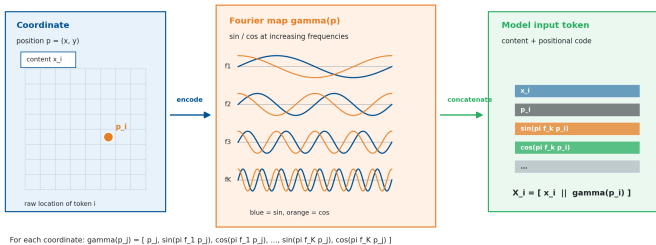
- Iterative refinement without instantiating a new processor at each step.
- Parameter count stays fixed as depth grows; computation stays in the N -slot latent space.

Fourier Positional Encoding

Perceiver receives a flat array, so location has to be part of each input token.

Fourier features turn position into a multiscale signature

The input stays a flat array: every element receives content plus its encoded coordinates.



General recipe

Use domain coordinates: images (u, v) , point clouds (x, y, z) , or text index t ; sine/cosine bands encode them at multiple scales.

Input seen by attention

The array is still flat, but each token is enriched before projection: $X_i = [x_i \parallel \gamma(p_i)]$.

Perceiver Forward Pass: Shape Checkpoints

Data flow in the original Perceiver:

- 1 Input array: each token stacks content x with a Fourier position code $\gamma(p)$,

$$X = [x \parallel \gamma(p)] \in \mathbb{R}^{M \times C}.$$

- 2 Learned latent array:

$$Z_0 \in \mathbb{R}^{N \times D}, \quad N \ll M$$

- 3 Cross-attention reads input:

$$Q = ZW_Q, \quad K = XW_K, \quad V = XW_V$$

- 4 Latent Transformer applies self-attention + FFN only on N latents.

Shape checkpoints:

Object	Shape	Meaning
X	$M \times C$	input array (content+pos)
Z_0	$N \times D$	learned latents
Q	$N \times d_k$	latent queries
K, V	$M \times d_k$	input keys/values
A	$N \times M$	attention map

Core idea

The input size M affects only the cross-attention read; depth is spent in the compact latent space.

Original Perceiver Output: Pooling Classifier

Fixed-shape output head:

$$Z_T = \text{Encoder}(X, Z_0)$$

$$z = \frac{1}{N} \sum_{i=1}^N Z_T[i]$$

$$\text{logits} = zW_{\text{cls}} + b_{\text{cls}}$$

- Mean pooling collapses all latents to one vector
- The classifier predicts one global label
- This is ideal for classification, not dense structured outputs

Where FFN and sharing fit:

- Each latent block is pre-norm attention, residual, FFN, residual
- FFN adds non-linearity after attention mixing
- Weight sharing repeats the processor over iterations

Fixed-shape output

Pooling collapses the N latents into a single vector, so this head emits one global prediction per input.

Computational Complexity Analysis

Model	Self-Attn	Cross-Attn	Total
Standard Transformer	$\mathcal{O}(M^2 d)$	—	$\mathcal{O}(M^2 d \cdot L)$
Perceiver	$\mathcal{O}(N^2 d)$	$\mathcal{O}(MNd)$	$\mathcal{O}((MN + N^2 L) d)$

Symbols:

- M : number of input elements
- N : number of latent slots, with $N \ll M$
- L : number of latent processing layers
- d : hidden channel dimension

Interpretation:

- The full input is read through cross-attention
- Repeated self-attention happens only in latent space
- If N is fixed, depth becomes independent of input length
- Concrete sizes are evaluated in the experiments

GELU Activation & LAMB Optimizer

GELU (Gaussian Error Linear Unit):

$$\text{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702 x)$$

- Smooth activation used in Transformer architectures
- Allows small negative gradient flow

LAMB (Layer-wise Adaptive Moments):

$$r_t^{(l)} = \frac{\|w_t^{(l)}\|}{\|w_t^{(l)} + \eta \hat{u}_t^{(l)}\|}$$

Per-layer trust ratio rescales updates and stabilizes large-batch training.

Why LAMB for Perceiver?

- Cross-attention bottleneck creates high variance gradients
- Standard Adam can **destabilize** with large batches
- LAMB safely scales batch sizes up to 1024

Optimizer	Large Batches	Layer-wise LR
Adam	Diverges	Global
LAMB	Stable	Per-layer trust

Implementation

Aspect	Paper implementation	Project code
Core model	Cross-attention into learned latents, latent self-attention, FFN, weight sharing	Same modules; weight sharing is exposed as an ablation switch
Model scale	Large vision/language runs: up to 512×1024 latents and 201M-param IO	Smaller runs: CIFAR-10 96×384 , ModelNet40/text 128×512
Data setting	ImageNet, ModelNet40, large language pretraining corpora	CIFAR-10, ModelNet40, WikiText-103 byte-level MLM, GLUE fine-tuning
Training budget	Distributed accelerators, batches up to 512–1024	Single-GPU budget, batches 32–64, LAMB retained for stability
Practical additions	Minimal regularization in the reference setup	Validation split carved from train, fixed seed, attention-map extraction, declarative run registry

The implementation keeps the architectural mechanism intact; the main differences are scale, data budget and training resources.

Implementation Differences from the Paper

Choice	Original paper (ImageNet)	This project
Input tokenization	Raw pixels, no patching ($M = 50,176$)	Raw pixels, no patching ($M = 1,024$ on 32×32)
Latent array $N \times D$	512×1024	96×384 (CIFAR-10), 128×512 (ModelNet40)
Cross-attend iterations T	8	4 (CIFAR-10), 2 (ModelNet40) — and T is itself an ablation axis
Positional encoding	2D Fourier, $K = 64$ bands/axis (258 dims)	Identical : 2D Fourier, $K = 64$ bands/axis, 258 dims + 3 RGB = 261
Weight sharing	On by default (shared across T)	Exposed as an ablation switch (on/off)
Model selection	—	Validation carved from train (45k/5k/10k); test touched once
Parameters	30–200M+	10.18M (CIFAR-10), 23.29M (ModelNet40), 18.87M (IO text)

Kept identical to the paper

Core mechanism preserved: cross-attention bottleneck $\rightarrow$ weight-shared latent transformer $\rightarrow$ mean-pooling (Perceiver) / output-query decoder (IO); GELU MLP, pre-norm LayerNorm, Fourier frequencies linearly spaced to Nyquist.

CIFAR-10: Experimental Setup

Goal: validate the Perceiver on raw pixels, with no convolutional assumption anywhere.

Hyperparameter	Value
Latent array ($N \times D$)	96×384
Cross-attn stages (T)	4, interleaved
Transformer blocks (L)	4 (shared)
Heads (cross / self)	1 / 8
Fourier bands / max freq	64 / 16
Dropout	0
Optimizer	LAMB
Learning rate	0.004
Scheduler	MultiStep 84/102/114
Batch size	64
Epochs	120, <i>no early stop</i>
Precision	bfloat16
Total parameters	10 175 362

Protocol — the part that makes the numbers usable

Train split: **45 000** images. Validation: **5 000**, carved out of the training set. Test: the official **10 000**, touched *once*, at the very end.

The reported checkpoint is the one with the best *validation* accuracy. Selecting the epoch on the same set you report on is **selection leakage** and inflates every number.

Data pipeline

- 32×32 RGB image $\rightarrow$ **1 024 pixels** as an unordered set
- Each pixel: 3 RGB + 2D Fourier PE = **261 dims** — the same per-pixel width as the paper
- No patching, no convolution, no learned tokenizer

The Noise Band: How Much Does a Number Move on Its Own?

Three runs, identical in everything but the seed, land 2.78 points apart. Read every other comparison against that.

Three runs, identical in everything but the seed:

Run	Seed	Test acc.
e01_baseline	42	71.63%
e31_baseline_seed1	1	68.85%
e32_baseline_seed2	2	70.97%
Spread (max – min)		2.78pp

The rule for every slide that follows

Any difference smaller than **2.78 percentage points** is not an effect. It is seed variance.

Why this slide exists

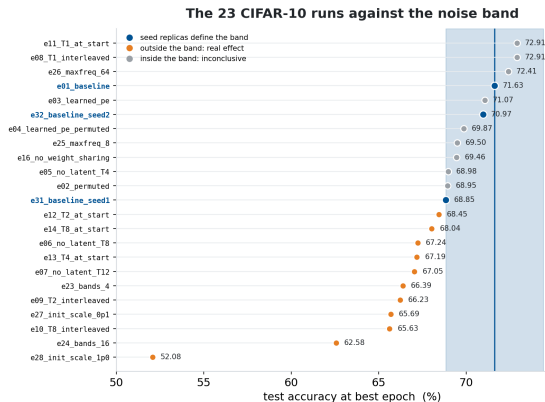
Two extra runs cost a few GPU-hours and change the status of all the others: without a measured band, every comparison is an opinion.

It also sets the honest ceiling of this study. On CIFAR-10 with 1 024 pixels and one GPU, most of the paper's ablations — measured on ImageNet with 50 176 pixels and 64 TPUs — simply do not have room to show up.

The band depends on the metric: 2.78pp on test accuracy, 2.94pp on best validation, 3.24pp on final validation. The conclusions do not change.

CIFAR-10: What Survives the Noise Band

Ranked against the band: twelve of the twenty-four runs clear it, the other twelve are inconclusive.



24 runs, 12 conclusive

- **Outside** the band:
positional encoding, latent init scale, cross-attend arrangement, Fourier bands, no latent transformer
- **Inside:** permutation, learned PE, weight sharing, max frequency

The chart plots 23 runs: e29_no_pe at 32.36% is off scale. Source: each run's results.json, test accuracy at the validation-selected epoch.

CIFAR-10: Permutation Invariance — the Headline Claim

Shuffling the pixels costs nothing measurable. Removing the encoding costs 39.27 points. That contrast is the claim.

Comparison	Positional encoding	Natural → permuted	Δ
e01 → e02	Fourier	71.63% → 68.95%	−2.68
e03 → e04	learned	71.07% → 69.87%	−1.20
e01 → e29	none at all	71.63% → 32.36%	−39.27
<i>Paper (ImageNet):</i> Fourier 78.0 → 78.0 learned 70.9 → 70.9			

What we measured

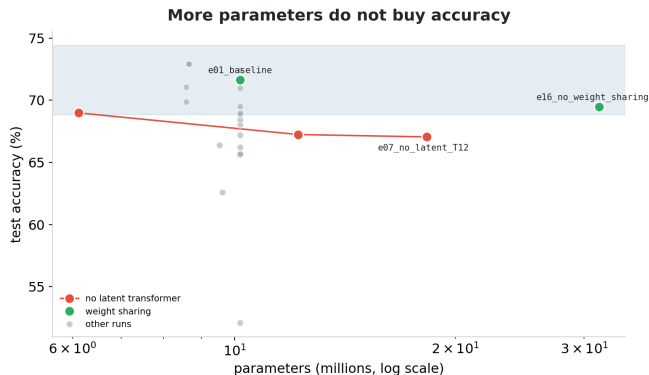
Both drops fall **inside** the 2.78pp band: shuffling the pixels causes no measurable damage. Invariance confirmed — in the honest, weak form: *we did not measure a loss*, which is not the same as proving the loss is zero.

The control that makes it a claim

Shuffling the pixels costs nothing measurable; **removing the encoding costs 39.27pp**. The same architecture that ignores the order collapses without the coordinates. Position enters through the **encoding**, never through the grid — and that contrast, not either number alone, is the evidence.

CIFAR-10: Capacity Does Not Buy Accuracy

Three times the parameters do not buy a point: capacity is not what limits this model at CIFAR-10 scale.



Weight sharing is free

- e01 71.63% with 10.2M params
- e16 69.46% with **31.5M** params
- $3.1\times$ the weights, **-2.17pp** — inside the band

Where you put capacity matters

- Remove the latent transformer, add cross-attends instead:
- 6.1M $\rightarrow$ 68.98%
- 12.2M $\rightarrow$ 67.24%
- 18.3M $\rightarrow$ 67.05%
- **More parameters, less accuracy**

CIFAR-10: How Many Cross-Attends, and Where

T	interleaved	all at start	Δ (arrangement)
1	72.91%	72.91%	0.00
2	66.23%	68.45%	+2.22
4	71.63% (baseline)	67.19%	−4.44
8	65.63%	68.04%	+2.41

The one conclusive result

At $T = 4$, interleaving beats front-loading by **4.44pp** — outside the band. Re-reading the input *after* the latents have been refined lets each pass look for something different.

Note the non-monotonicity along T : reading the input once is as good as reading it four times, with 15% fewer parameters.

A reproducibility check, for free

At $T = 1$ the two arrangements produce the *same computational graph*, and indeed give bit-identical results: 72.91%, epoch 90, 8 654 662 params. Fixed seed $\Rightarrow$ reproducible training.

CIFAR-10: The Hyperparameter That Actually Matters

Sweep	Value	Test acc.
Latent init σ	0.02	71.63%
	0.1	65.69%
	1.0	52.08%
Fourier bands	4	66.39%
	16	62.58%
	64	71.63%
Max frequency	8	69.50%
	16	71.63%
	64	72.41%

Latent initialisation: the only clean monotone effect

σ from 0.02 to 1.0 costs **19.55pp**. At $\sigma = 1.0$ the best epoch is **9 out of 120**: the model learns briefly, then degrades for the remaining 111 epochs.

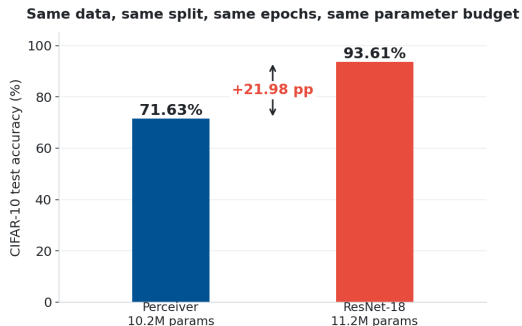
Two honest caveats

Bands is non-monotone (4 beats 16), and both low points are runs whose *final* validation collapsed (22.5% and 55.5%) — an optimisation problem, not a resolution one.

Max frequency sits entirely inside the noise band. The whole axis is inconclusive, and saying so is part of the result.

CIFAR-10: the Reference That Is Not a Perceiver

Neither Perceiver paper benchmarks CIFAR-10, so we trained the reference ourselves: a ResNet-18, same split, same epochs, same budget.



The uncomfortable comparison comes first

A **ResNet-18** on the same split, the same 120 epochs and 11.2M parameters against 10.2M reaches **93.61%**, where e01_baseline reaches **71.63%**: **-21.98pp**.

Neither Perceiver paper benchmarks CIFAR-10, so without this run there is nothing to read the number against.

What the gap actually measures

On ImageNet the paper *beats* ResNet-50 by +4.5pp (78.0 vs 73.5). What differs between the two settings is not the architecture, it is the **scale**.

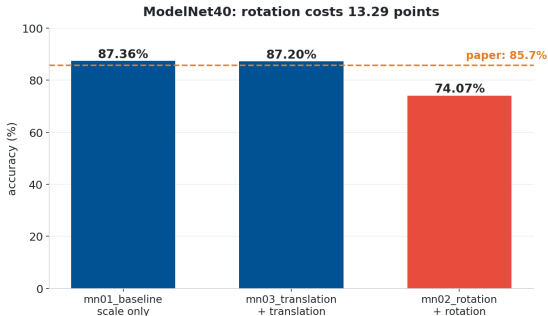
The Perceiver's claim was never accuracy — it is **having no prior on the domain**. Here that prior is worth 22 points.

ModelNet40: Same Network, 3D Point Clouds

The same network, fed 2,048 points in 3D instead of pixels. Only the positional encoding changes dimension.

Setting	Value
Latent array	128×512
Cross-attn / blocks	2 / 6
Points per model	2 048
Input per point	387 dims
Fourier bands / max f	64 / 1120
Optimizer / lr	LAMB / 0.001
Batch (paper: 512)	32
Epochs	120
Parameters	23 288 928

Metric note: ModelNet40 has no separate validation split — the standard test set (2 468 objects) is the validation set, so the figure is `val_accuracy` at the best epoch.



Above the paper

87.36% vs the paper's **85.7%**, with a batch $16\times$ smaller.
Plausible: ModelNet40 is small and its objects arrive canonically aligned, so the large-batch advantage that dominates ImageNet counts for little here.

ModelNet40: Rotation Costs 13 Points — and Why

Translation is free. Rotation costs 13.29 points — and the reason is the positional encoding again.

Augmentation	Accuracy	Δ
scale only	87.36%	—
+ translation	87.20%	−0.16
+ rotation	74.07%	−13.29

The largest effect in the whole project

And the one with the cleanest theoretical explanation.

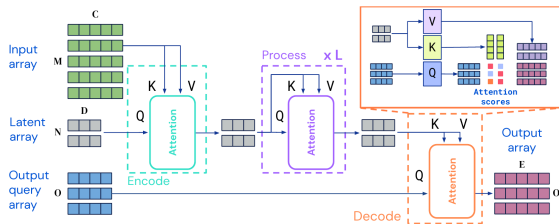
The Perceiver has no rotational inductive bias

3D Fourier PE encodes **absolute** position. Rotating an object changes every coordinate — and therefore every encoding — while the label stays the same. The network must learn from scratch an invariance the architecture does not grant, and that this task does not even need: ModelNet40 objects are already canonically aligned.

Translation is nearly harmless (−0.16) because it shifts all points equally: the *relative* structure survives.

Takeaway: a missing inductive bias cannot be patched with augmentation.

Perceiver IO: Structured Outputs



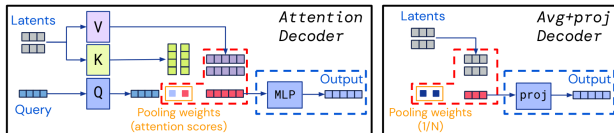
Extension over Perceiver (Jaegle et al., ICLR 2022):

- **Encode:** input $\rightarrow$ latent (same cross-attn encoder)
- **Process:** latent self-attention ($\times L$)
- **Decode:** output queries read latents

Why it matters:

- Original Perceiver: one pooled global output
- IO: task-specific output shape
- Same latent bottleneck, more general decoder

Perceiver IO Decoder with Output Queries



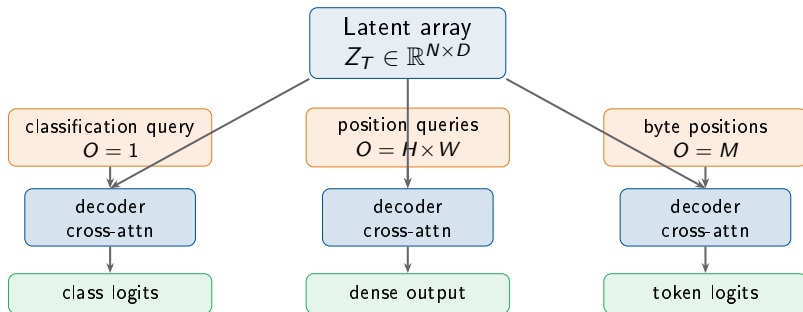
Decoder cross-attention (single layer):

$$Q_{\text{dec}} = \text{OutputQuery } W_Q, \quad K_{\text{dec}} = \text{Latent } W_K, \quad V_{\text{dec}} = \text{Latent } W_V$$

$$\text{Output} = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{dec}}^T}{\sqrt{d_k}}\right) V_{\text{dec}} \in \mathbb{R}^{O \times D}$$

- A task head then projects each of the O rows to the output channels E (logits, pixels, bytes)
- The number of output queries O is **task-dependent** and independent of N and M
- A single decoder cross-attention layer suffices (no self-attention among outputs)

Output Query Design per Task



Classification

One learned query produces one global prediction.

Dense outputs

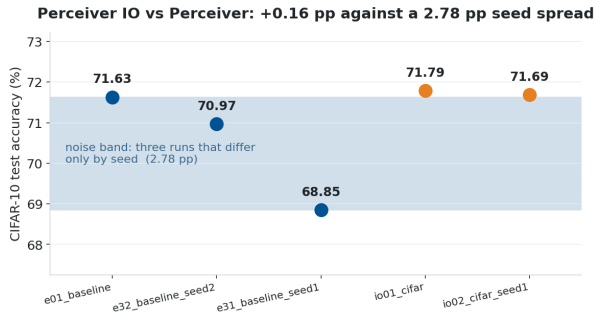
Output shape is controlled by the query array.

Masked LM

One query per byte; the loss counts only the masked ones.

Perceiver IO on CIFAR-10: the Decoder Changes Nothing Here

On a single label out of ten, the query decoder and mean pooling are the same thing — and the measurement says so.



What changed, and what did not

Same encoder as e01_baseline; the mean-pooling head is replaced by a **decoder with one output query**, and the training recipe follows the IO paper (App. A.1): $\text{lr } 10^{-3}$, cosine.

io01_cifar **71.79%** vs e01 **71.63%**: +0.16pp, against a seed spread of **2.78pp**. The replica io02 lands at 71.69%.

This was the expected outcome

No difference is the result, not a disappointment. The decoder earns its keep when the output is **structured** — one query per byte in the MLM, one per task in the multitask run. For one label out of ten, querying the latents and averaging them are the same thing.

WikiText-103: Byte-Level Masked Language Model

Pre-train a language encoder with no tokenizer at all: the input is the raw UTF-8 byte stream.

Setting	Value
Latent array	128×512
Cross-attn / blocks	1 / 4
Heads (cross / self)	1 / 8
Sequence length	512 bytes
Vocabulary	256 + 1 mask
Masking rate	15%
Output queries	one per position
Optimizer / lr	LAMB / 0.001
Epochs / batch	10 / 32
Parameters	18 872 740
Masked-byte accuracy	86.68%
Chance level (1/256)	0.39%

Where the decoder finally earns its keep

~222× **better than chance**, reading raw UTF-8 with **no tokenizer at all**. Nearly nine masked bytes out of ten reconstructed.

Why bytes are affordable here

Cross-attention costs $O(M \cdot N)$ — *linear* in sequence length — instead of $O(M^2)$. Going from 512 tokens to 2 048 bytes costs BERT 16× in attention, and Perceiver IO only 4×.

Read this number carefully

Masked-byte accuracy is not comparable to masked-token accuracy: the space is 256, not tens of thousands. It shows the pre-training worked — the comparison with BERT lives on GLUE.

GLUE: Eight Tasks, One Encoder, No Tokenizer

The same encoder, the same weights, one query swapped — and two of the eight numbers turn out not to be learning.

Run	Task	Budget	Result
io_glue_colab	acceptability	30	69.13
io_glue_sst2	sentiment	10	80.73
io_glue_mrpc	paraphrase	30	70.59
io_glue_stsb	similarity (reg.)	30	60.82
io_glue_qqp	duplicate Q	3	84.66
io_glue_mnli	NLI (3-way)	3	65.80
io_glue_qnli	QA entailment	10	80.65
io_glue_rte	entailment	30	56.68
Mean over the eight			71.13
Paper, Tab. 1 (byte-level)			81.0

STS-B is Pearson $\times 100$, the official GLUE metric. Grey marks the two majority-class predictors. Fine-tuning starts from the MLM checkpoint; the epoch budget follows task size (393 k examples against 2.5 k).

Two of these numbers are not learning

CoLA scores 721 correct out of 1 043 — *exactly* the majority-class count. **MRPC** beats its majority by 9 examples out of 408. Both are constant predictors wearing an accuracy.

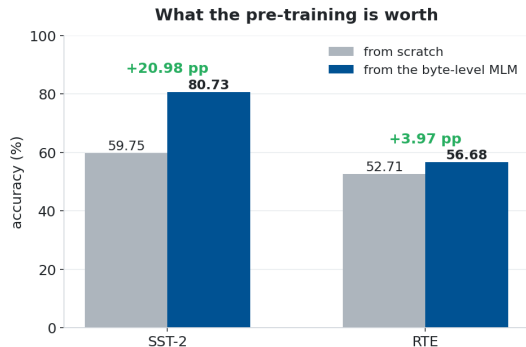
The mean of 71.13 therefore rests on **six** tasks, not eight. CoLA in the paper is scored with Matthews' correlation, which for a constant predictor is **0**.

The gap that does not close

71.13 against the paper's 81.0. The scale is not comparable: **18.9M** parameters pre-trained on WikiText-103 for 10 epochs, against **201M** on Wikipedia + C4 for 500 000 steps.

GLUE: What the Pre-training Is Actually Worth

Two tasks fine-tuned twice — once from the MLM checkpoint, once from random weights. Everything else identical.



Why two controls and not eight

A fine-tuned 80% means nothing on its own — maybe the task is solvable at 80% from random weights. The two scratch runs are chosen at the **extremes** of data size: **SST-2** has 67 000 training examples, **RTE** has 2 500. Everything else is identical.

And the prediction holds

SST-2: 59.75% → 80.73%, +20.98pp. RTE: 52.71% → 56.68%, +3.97pp.

Transfer is worth five times more where the data abounds. The small task cannot exploit the encoder it was given: 2 500 examples are not enough to move it far from its starting point.

GLUE Multitask: the Comparison That Replicates Best

One model with eight output queries against eight separate fine-tunings — the comparison from Table 2 of the IO paper.

One model with eight output queries beats eight models: 74.05 vs 71.13

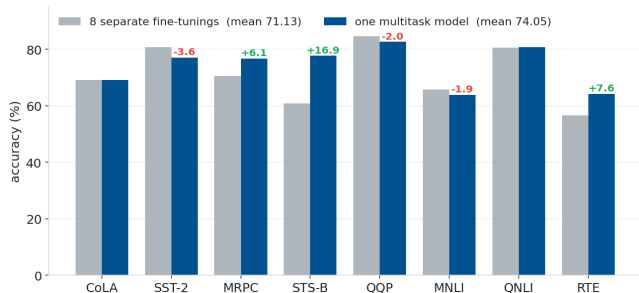


Table 2 of the IO paper

One model with **one output query per task**, against eight separate fine-tunings. Paper: 81.8 vs 81.0, +0.8. Here: 74.05 vs 71.13, **+2.92**.

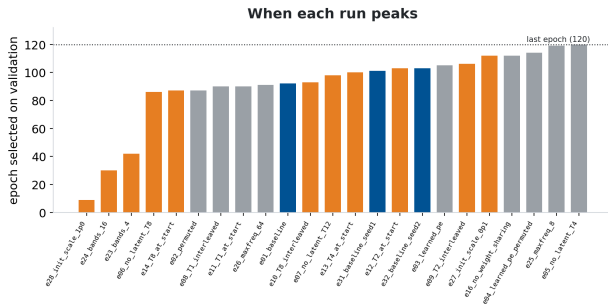
The absolute level stays ~8 points below. But **sign and order of magnitude hold** — and that relation, not the level, is what Tab. 2 claims.

Not a fair fight — in our favour

The multitask model selects **one** epoch for all eight tasks; the separate models select **eight**. It wins anyway — and a ninth task would cost **one query**, not another model.

Convergence: When Each Run Peaks

There is no early stopping: every run trains the full schedule and reports the checkpoint that was best on validation.



Why this matters

The distribution of best epochs is itself a diagnostic: a run that peaks early and then decays is unstable, whatever its best number says.

Three unstable runs

Validation accuracy, best epoch → final epoch:

e23: 67.74% @42 → **22.54%**

e24: 62.66% @30 → **55.46%**

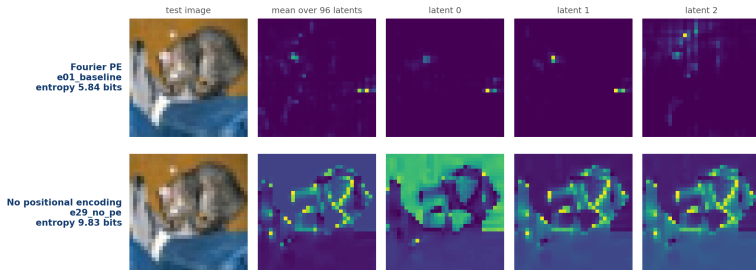
e28: 53.36% @9 → 46.32%

Their reported numbers come from the validation-selected checkpoint — the correct methodology — but the instability has to be declared, not hidden.

e05_no_latent_T4 peaks at epoch **120**: it was still improving when the schedule ended.

Attention Maps: Where the Latents Look

With coordinates the latents pick locations; without, they can only pick colours.



Fourier PE (e01, 71.63%)

Each latent attends to a few pixels at specific locations: mean entropy **5.84 bits** over 1 024 pixels (uniform: 10 bits). Latents specialise by region.

No PE (e29, 32.36%)

Same-colour pixels are indistinguishable: attention follows colour boundaries and spreads over the image — **9.83 bits**, 98% of uniform.

Comprehensive Results Summary

Forty-two runs, all completed. Every number below traces to a `results.json` — none is quoted from memory.

Modality	Model	Best Acc.	Reference	Experiments
Images (CIFAR-10)	Perceiver	72.91%	93.61% [†]	24 runs
Images (CIFAR-10)	Perceiver IO	71.79%	71.63% (Perceiver)	2
Point Clouds (ModelNet40)	Perceiver	87.36%	85.7%	3 augmentations
Text (WikiText-103)	Perceiver IO	86.68% MLM	0.39% chance	1
NLU (GLUE, 8 tasks)	Perceiver IO	71.13 mean	81.0 (paper)	11
Images (CIFAR-10)	ResNet-18	93.61%	—	1

[†] A ResNet-18 trained here under identical conditions. Neither Perceiver paper benchmarks CIFAR-10, so the reference had to be measured rather than quoted.

What is measured

- ModelNet40 **beats** the paper: +1.66pp
- Byte-level MLM works: 86.68% against a 0.39% chance level
- One multitask model beats eight separate ones: 74.05 vs 71.13
- Noise band **2.78pp**, measured on 3 seed replicas
- 12 of 24 CIFAR runs are conclusive

Scope of the campaign

- Declarative registry of **42 runs**, **42** completed
- 3 modalities: 2D images, 3D point clouds, raw bytes
- One command per run: `experiments.py -run <id>`
- Every figure above traces to a `results.json`; none is quoted from memory

Gap vs Original Papers

On the modality with the weakest domain prior we beat the paper. On the one with the strongest, a convolutional net of the same size beats us.

ModelNet40 (Point Clouds):

Source	Acc.
Original Paper	85.7%
This implementation	87.36%
Gap	+1.66pp

We are above the paper.

With a batch $16\times$ smaller (32 vs 512). Plausible because ModelNet40 is small and canonically aligned, so the large-batch advantage counts for little.

What the two columns say together

On the modality with a **weak** domain prior — 3D point clouds — we are above the paper. On the one with a **strong** prior, a convolutional net of the same size wins by 22 points. On ImageNet the paper beats ResNet-50 by +4.5pp: the prior stops paying at scale, and **that is the claim being replicated**, not an absolute accuracy.

CIFAR-10 (Images):

Source	Acc.
ResNet-18, same budget	93.61%
e01_baseline	71.63%
Gap	−21.98pp

The reference had to be measured.

Neither Perceiver paper benchmarks CIFAR-10, so a convolutional net was trained here under identical conditions rather than quoting a number from elsewhere.

Hardware Limitations & Design Choices

Every limitation below is one GPU instead of 512 TPU cores. The architecture is the paper's; the budget is not.

Parameter	Original Paper (Google)	Project Setup
Hardware	512 TPU v3 cores	1 NVIDIA RTX 3080
Batch size	up to 1024	32–64
Latents (N)	256–512	96–128
Model size	30–200M+ params	10.2–23.3M params
Pre-training corpus	Wikipedia + C4, 201M-param IO	WikiText-103, 18.9M-param IO

Consequences:

- LAMB at batch 32–64, not the 512–1024 it targets
- 96–128 latents instead of 512
- One seed per ablation, three for the baseline
- No search beyond the registered ablations

Mitigation strategies:

- Weight sharing: 10.2M parameters, not 31.5M
- bf16 autocast + gradient clipping (LAMB at lr 0.004)
- A measured noise band, not more runs per cell
- Validation carved from train; test touched once

Possible Improvements

Architecture:

- Increase latent count ($N = 256+$) with gradient checkpointing
- Add multi-scale cross-attention (coarse $\rightarrow$ fine)
- Explore relative positional encodings (RoPE, ALiBi)
- Deeper decoder for structured outputs

Training:

- Larger batch sizes with gradient accumulation
- More seeds per ablation, to narrow the 2.78pp band
- Longer training schedules
- Advanced augmentation (CutMix, MixUp)

Data & Tasks:

- ImageNet pre-training for vision
- Longer byte-level pre-training: more epochs, larger corpus
- Multi-modal joint training
- Audio modality experiments

Analysis:

- Layer-wise attention probing
- Latent space clustering visualization
- Systematic hyperparameter search
- Comparison with efficient Transformers (Linformer, Performer)

Empirical Summary

Everything measured, split by whether it survived the noise band.

Conclusive (outside the 2.78pp band):

- **Positional encoding** is the whole thing: removing it costs **39.27pp** (71.63→32.36%)
- **Generality**: one architecture, 87.36% on 3D point clouds — above the paper — and 86.68% on raw bytes
- **Pre-training transfers**: SST-2 59.75→80.73%, and 5× less on the small task
- **Latent init scale** is the critical hyperparameter: 0.02→71.63%, 1.0→52.08%
- **Interleaving** cross-attends beats front-loading by 4.44pp
- **Rotation augmentation** costs 13.29pp on ModelNet40

Central methodological finding

Measuring the **noise band** — three replicas differing only by seed — cost two runs and changed the status of all the others. Without it, 12 of the 24 comparisons would have been reported as effects when they are variance.

Inside the band — “no measurable damage”:

- Permutation invariance, with *both* positional encodings
- Weight sharing: 3.1× fewer parameters at no measurable cost
- Perceiver IO vs Perceiver on images: +0.16pp

Measured, and not in our favour:

- A ResNet-18 on the same budget: **93.61%** vs 71.63%
- GLUE mean 71.13 against the paper's 81.0
- Two GLUE tasks (CoLA, MRPC) are majority-class predictors
- No configuration beats the baseline outside the band — every conclusive effect is **negative**

What the architecture gives, and what it still costs at this scale.

Architecture properties:

- Single architecture for images, point clouds, and text
- Linear complexity in input size
- Weight sharing: $3.1\times$ fewer parameters at no measurable cost
- Zero CNN/RNN domain-specific assumptions
- Output queries: one more task costs **one query**, not another model

Remaining limitations:

- At this scale the domain prior is worth **22 points** (93.61 vs 71.63)
- Entirely dependent on the positional encoding (-39.27pp without it)
- GLUE sits ~ 10 points below the paper (71.1 vs 81.0), at 1/10 of the parameters and pre-training
- LAMB at batch 64 is sensitive to the latent init scale
- Half the image ablations are indistinguishable from noise

Closing statement

A **single general-purpose architecture** does apply to images, point clouds and raw bytes — and on the modality with the weakest domain prior it beats the paper. Its advantage was never accuracy: it is **having no prior on the domain**. At CIFAR-10 scale that prior is worth 22 points; the paper's own claim is that it stops paying at ImageNet scale, and nothing measured here contradicts it.

Thank you!

Questions?

Francesco Gigli and Corso Vignoli

Code and reproduction instructions shared with the course